

CSC343H5 Lecture 5

...

Naaz Sibia (naaz.sibia@utoronto.ca)

Admin Notes: Checkpoint 3 is Out

- Five queries
 - 3 have SQL queries + Relational Algebra
 - 2 are only about SQL
- Testing script attached
 - But I will test with other datasets :)

Anonymous Feedback

Hi Prof. Naaz,

If we have time, could we briefly go through the solutions for the PCRS Week 2 exercises? I found some of the questions a little ambiguous, and the AI feedback from DUET was not very helpful.

Thanks so much.

Yes – I will try to upload a solution set with explanations!

Also - just Naaz :)

Grading Timeline - Checkpoint 2

- Starting today - will try to get it back to you by the end of next week :)
- If your DDL passes the given autotests - it's fine to use in Checkpoint 3.
 - If not - email me so we can fix the issue!

Recap Quiz time!

New Content!



Example. Spotify Playlists



Let's take these tables -

song_id	title	artist
1	Blinding Lights	The Weeknd
2	Levitating	Dua Lipa
3	Peaches	Justin Bieber

play_id	user_id	song_id	played_at
1	u1	1	2024-01-01
2	u1	1	2024-01-02
3	u2	2	2024-01-01

Example. Spotify Playlists



Let's take these tables -

song_id	title	artist
1	Blinding Lights	The Weeknd
2	Levitating	Dua Lipa
3	Peaches	Justin Bieber

play_id	user_id	song_id	played_at
1	u1	1	2024-01-01
2	u1	1	2024-01-02
3	u2	2	2024-01-01

- How do we find which songs u1 listened to?



* credits to my brother for this picture

Example. Spotify Playlists



Let's take these tables -

song_id	title	artist
1	Blinding Lights	The Weeknd
2	Levitating	Dua Lipa
3	Peaches	Justin Bieber

play_id	user_id	song_id	played_at
1	u1	1	2024-01-01
2	u1	1	2024-01-02
3	u2	2	2024-01-01

- How many times was each song played?



* credits to my brother for this picture

Example. Spotify Playlists



Let's take these tables -

song_id	title	artist
1	Blinding Lights	The Weeknd
2	Levitating	Dua Lipa
3	Peaches	Justin Bieber

play_id	user_id	song_id	played_at
1	u1	1	2024-01-01
2	u1	1	2024-01-02
3	u2	2	2024-01-01

- How many times was each song played?



* credits to my brother for this picture

Example. Spotify Playlists



Let's take these tables -

song_id	title	artist
1	Blinding Lights	The Weeknd
2	Levitating	Dua Lipa
3	Peaches	Justin Bieber

play_id	user_id	song_id	played_at
1	u1	1	2024-01-01
2	u1	1	2024-01-02
3	u2	2	2024-01-01

- Which songs have never been played?



* credits to my brother for this picture

Joins



The JOIN clause is used to combine rows from two or more tables, based on a related column between them.

Types:

- Union/Full Join
- Outer Join
- Inner Join
- Right Join
- Left Join

Joins



- The match condition is the bridge between tables, in our case, `song_id` appears in both.

song_id	title	artist
1	Blinding Lights	The Weeknd
2	Levitating	Dua Lipa
3	Peaches	Justin Bieber

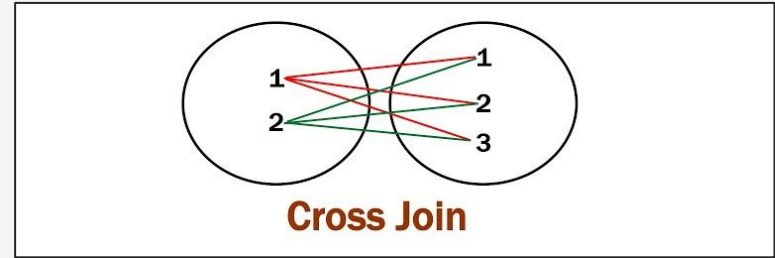
play_id	user_id	song_id	played_at
1	u1	1	2024-01-01
2	u1	1	2024-01-02
3	u2	2	2024-01-01



Think of it as: take every play, look up its song, and glue the row together.

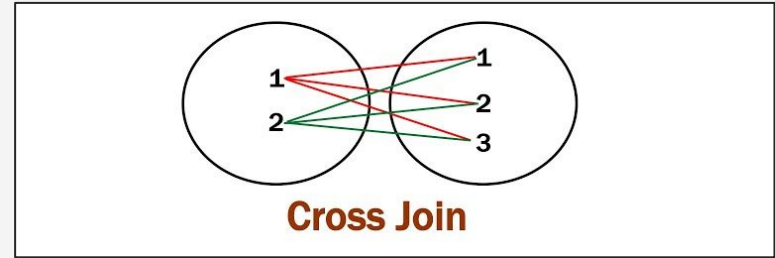
Cross Product/Join

A cross join returns **every combination** of rows from two tables, no ON condition, no matching key. If the left table has 4 rows and the right table has 4 rows, you get 16 rows.



Cross Product/Join

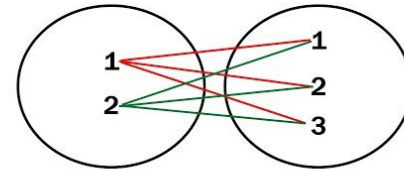
A cross join returns **every combination** of rows from two tables, no ON condition, no matching key. If the left table has 4 rows and the right table has 4 rows, you get 16 rows.



A missing ON clause in an old-style implicit join (FROM a, b) is a cross join.

Cross Product/Join

A cross join returns **every combination** of rows from two tables, no ON condition, no matching key. If the left table has 4 rows and the right table has 4 rows, you get 16 rows.



Cross Join

A cross join on two large tables is one of the easiest ways to accidentally bring a database to its knees. $1,000 \text{ rows} \times 1,000 \text{ rows} = 1,000,000 \text{ rows}$. It grows fast.



Cross Product/Join

```
SELECT color, size  
FROM Colors CROSS JOIN Sizes;
```

① Step 1

Colors

color
Red
Blue

Red will pair with all sizes

Blue will pair with all sizes



② Step 2

Sizes

size
Small
Medium
Large

③ Step 3

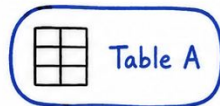
Result

color	size
Red	Small
Red	Medium
Red	Large
Blue	Small
Blue	Medium
Blue	Large

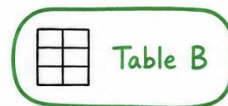
Red paired with all sizes

Blue paired with all sizes

$2 \times 3 = 6$ rows



+



→



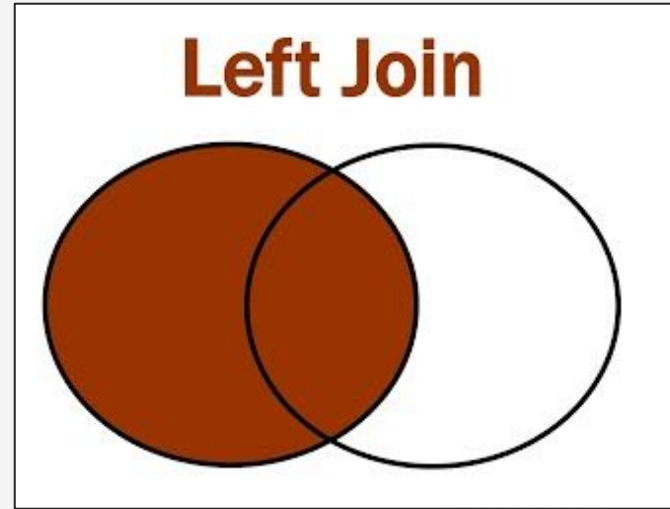
→



Left (Outer) Join

All rows from the left, NULLs where no match on right.

Outer is an optional keyword, a Left Join is outer by default.



Left Outer Join

```
SELECT Students.name, Clubs.club_name
FROM Students LEFT JOIN Clubs
ON Students.club_id = Clubs.club_id;
```

① Left table: Students

sid	name	club_id
1	Ava	10
2	Ben	20
3	Chloe	30
4	Diego	NULL

② Right table: Clubs

club_id	club_name
10	Chess
20	Robotics
40	Music

③ LEFT JOIN result

name	club_name
Ava	Chess
Ben	Robotics
Chloe	NULL
Diego	NULL

★ All rows from the **LEFT** table stay.

★ If no match on the right, fill with **NULL**.

LEFT table

+

matching RIGHT data



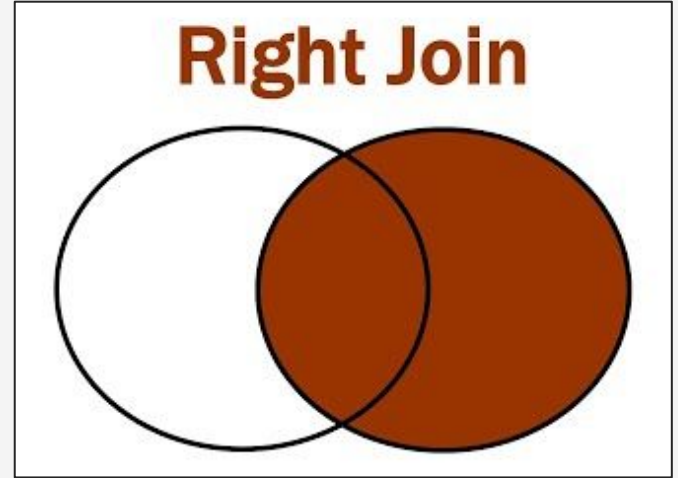
unmatched RIGHT values become NULL

...		NULL
...		NULL

Right Outer Join

All rows from the right, NULLs where no match on right.

Outer is an optional keyword, a Left Join is outer by default.



Right Outer Join

```
SELECT Students.name, Clubs.club_name
FROM Clubs RIGHT JOIN Students
ON Clubs.club_id = Students.club_id;
```

① Left table: Clubs

club_id	club_name
10	Chess
20	Robotics
40	Music

② Right table: Students

sid	name	club_id
1	Ava	10
2	Ben	20
3	Chloe	30
4	Diego	NULL

③ RIGHT JOIN result

name	club_name
Ava	Chess
Ben	Robotics
Chloe	NULL
Diego	NULL

★ All rows from the **RIGHT** table stay.

★ If no match on the **LEFT**, fill with **NULL**.

LEFT table

+

RIGHT table

→

Keep all RIGHT rows

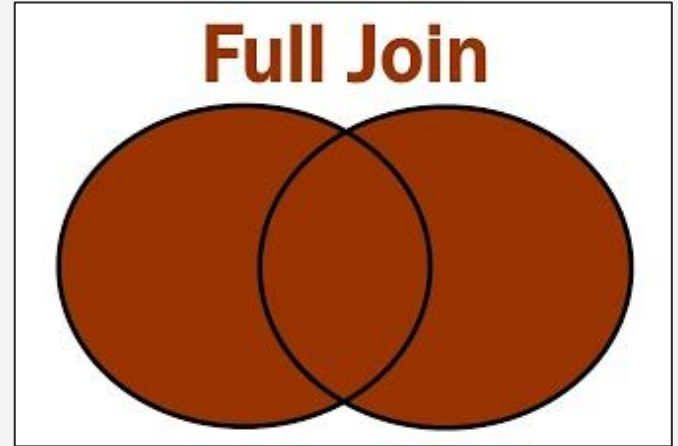
→

Unmatched LEFT values become NULL

...		NULL
...		NULL

Full Outer Join

All rows from both sides, NULLs where no match.



Full Outer Join

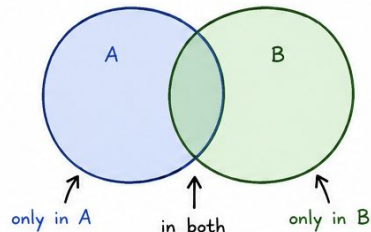
Tables

A (Students)

id	name
1	Alice
2	Bob
3	Cara

B (Enrollments)

student_id	course
2	Math
2	DB
4	AI



SQL Syntax

```
SELECT *  
FROM A  
FULL OUTER JOIN B  
ON A.id = B.student_id;
```

Result

A.id	A.name	B.student_id	B.course
1	Alice	NULL	NULL
2	Bob	2	Math
2	Bob	2	DB
3	Cara	NULL	NULL
NULL	NULL	4	AI

← from A, no match in B

} matches in both
(Bob has two enrollments)

← from A, no match in B

← from B, no match in A

Key Idea

- Rows only in A appear with NULLs for B columns.
- Rows only in B appear with NULLs for A columns.
- Rows in both tables show the matched data.



FULL OUTER JOIN = (LEFT JOIN) UNION (RIGHT JOIN)

Inner Join

```
SELECT Students.name, Clubs.club_name
FROM Students INNER JOIN Clubs
ON Students.club_id = Clubs.club_id;
```

① Left table: Students

sid	name	club_id
1	Ava	10
2	Ben	20
3	Chloe	30
4	Diego	NULL

→ = match found

---> X = no match

② Right table: Clubs

club_id	club_name
10	Chess
20	Robotics
40	Music

↑
No match in Students

③ INNER JOIN result

name	club_name
Ava	Chess
Ben	Robotics

★ Only rows that match in BOTH tables stay.

LEFT table

+

RIGHT table

→

Keep matches only



Unmatched rows are removed.

Exercise

Relational Algebra

Relational algebra is a set of operations that take one or two relations as input and produce a new relation as output.

Operators	Description
σ (Selection) - Unary	Filter tuples based on a condition
π (Projection) - Unary	Keep only specified attributes
ρ (Rename) - Unary	Rename relation and/or attributes
$\cup \cap -$ (Set Ops) - Binary	Set union, intersection, difference
$\times$ (Cross Product) - Binary	Cartesian product of two relations
$\bowtie$ (Join) - Binary	Cross product + selection condition

Relational Algebra

Why relational algebra?

- **Foundational:** Learning relational algebra provides the mathematical and logical foundation for how relational databases (like SQL) process, filter, and retrieve data. It equips you with the mental framework to write, debug, and optimize complex queries without relying on trial-and-error.
- **Abstraction:** SQL is the practical implementation of relational algebra. Concepts like SELECT, WHERE, and JOIN map directly to algebraic operators.
- **Execution:** When a database engine receives an SQL query, it translates it into a relational algebra execution plan.
- **Optimization:** Can help optimize queries

Schema vs. Instance

Schema (the structure)

A relation schema $R(A_1, A_2, \dots, A_n)$ defines:

- Name of the relation
- Set of attributes $A_1 \dots A_n$
- Domain (type) of each attribute

Example:

```
Students(sid: INT,  
         name: VARCHAR,  
         major: VARCHAR,  
         gpa: FLOAT)
```

The schema is like a class definition, it describes the structure but contains no data.

Instance (the data)

An instance is a finite set of tuples that conform to the schema.

Formally: $r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$

Example (current state):

sid	name	major	gpa
1	Anya	CS	3.8
2	Boris	Math	3.2
3	Carmen	CS	3.9
4	Diego	Bio	3.1

The instance changes over time. The schema stays fixed.

UNARY OPERATIONS

Selection, Projection, Rename

SELECTION (σ) - THEORY

Syntax: $\sigma_{condition}(R)$

Result: $\{ t \in R \mid condition(t) \}$

Properties

- Result has the same schema (columns)
- Result is a subset of R's tuples
- Selection is commutative:

$$\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$$

- Conjunction:

$$\sigma_{c1 \wedge c2}(R) = \sigma_{c1}(\sigma_{c2}(R))$$

- Disjunction:

$$\sigma_{c1 \vee c2}(R) = \sigma_{c1}(R) \cup \sigma_{c2}(R)$$

Condition Types

Simple

$gpa > 3.5$

Compare attribute to constant

Attribute comparison

$salary > budget$

Compare two attributes

Conjunction (AND)

$gpa > 3.5 \wedge major = 'CS'$

Both conditions must hold

Disjunction (OR)

$gpa > 3.9 \vee major = 'CS'$

Either condition holds

Negation (NOT)

$\neg(major = 'CS')$

Condition must be false

SELECTION (σ) - WORKED EXAMPLE

Students

sid	name	major	gpa
1	Anya	CS	3.8
2	Boris	Math	3.2
3	Carmen	CS	3.9
4	Diego	Bio	3.1
5	Elif	CS	3.7

Simple condition

$$\sigma_{\text{gpa}} > 3.5(\text{Students})$$

Result: Anya (3.8), Carmen (3.9), Elif (3.7)

Conjunction (AND)

$$\sigma_{\text{major} = \text{'CS'} \wedge \text{gpa} > 3.7}(\text{Students})$$

Result: Anya (3.8), Carmen (3.9)

Disjunction (OR)

$$\sigma_{\text{major} = \text{'CS'} \vee \text{gpa} > 3.8}(\text{Students})$$

Result: Anya, Carmen, Elif

** Boris satisfies gpa > 3.8? No, he satisfies major='CS'? No. So not included.*

PROJECTION (π) - THEORY

Syntax: $\pi_{A_1, A_2, \dots, A_n}(R)$ **Result:** $\{ t[A_1, \dots, A_n] \mid t \in R \}$

Properties

- Result schema has only the listed attributes
- Duplicate tuples are ELIMINATED, result is a set
- If $A_1 \dots A_n$ includes a key, no duplicates exist
- π is NOT commutative in general
- $\pi_A(\pi_B(R)) = \pi_A(R)$ if $A \subseteq B$ (cascade rule)
- $\pi_A(\sigma_c(R)) = \sigma_c(\pi_A(R))$ only if c uses only A 's attributes

What projection does:

sid	name	major	gpa
1	Anya	CS	3.8
2	Boris	Math	3.2
3	Carmen	CS	3.9

$\pi_{\text{name}}(\text{Students})$

name
Anya
Boris
Carmen



π removes duplicates, unlike SQL's SELECT which preserves them. Use SELECT DISTINCT in SQL to match π behaviour.

RENAME (ρ) - THEORY & USAGE

Syntax: $\rho_S(R)$ rename relation $\rho_S(B_1, \dots, B_n)(R)$ - rename relation and all attributes

- Cases:
 - "Find pairs of students in the same major"
 - Need TWO copies of Students to compare rows. Without rename, columns would clash:
 - $\rho_{S1}(\text{Students}) \bowtie \{S1.\text{major}=S2.\text{major}\} \rho_{S2}(\text{Students})$
S1 and S2 are now distinct relation names
 - Give a meaningful name to a subexpression:
 - $\rho_{\text{HighGPA}}(\sigma_{\text{gpa} > 3.5}(\text{Students}))$
 - Now use HighGPA like any other relation in subsequent operations.
 - $\text{HighGPA} \bowtie \text{Takes}$

SET OPERATIONS

Union, Intersection, Difference

UNION COMPATIBILITY - A PREREQUISITE

All three set operations ($\cup$, $\cap$, $-$) require the two relations to be union-compatible:

1

Same number of attributes

R and S must have the same arity (number of columns).

2

Compatible domains

Corresponding attributes must have compatible types (e.g., both INT, or both VARCHAR).

3

Names don't have to match

Attribute names are inherited from the LEFT relation R in $R \cup S$, $R \cap S$, $R - S$.

✓ Compatible

Students(sid,name,gpa) $\cup$ Graduates(id,fullName,score)
Same arity, compatible types - OK!

✗ NOT Compatible

Students(sid,name,gpa) $\cup$ Courses(cid,title,credits)
Different semantics - ERROR!

UNION (U)

R = CS students who took CSC108 S = students who took MAT136

R

name
Anya
Boris
Elif

S

name
Anya
Carmen
Elif
Diego

R U S Union → SQL: UNION

All names that appear in R or S (or both). Duplicates removed.

UNION (U)

R = CS students who took CSC108 S = students who took MAT136

R

name
Anya
Boris
Elif

S

name
Anya
Carmen
Elif
Diego

R U S Union → SQL: UNION

All names that appear in R or S (or both). Duplicates removed.

Result: {Anya, Boris, Carmen, Diego, Elif}

UNION (U)

R = CS students who took CSC108 S = students who took MAT136

R

name
Anya
Boris
Elif

S

name
Anya
Carmen
Elif
Diego

$R \cap S$ Intersection → SQL: INTERSECT

Names that appear in BOTH R and S.

UNION (U)

R = CS students who took CSC108 S = students who took MAT136

R

name
Anya
Boris
Elif

S

name
Anya
Carmen
Elif
Diego

$R \cap S$ Intersection $\rightarrow$ SQL: INTERSECT

Names that appear in BOTH R and S.

Result: {Anya, Elif}

UNION (U)

R = CS students who took CSC108 S = students who took MAT136

R	
name	
	Anya
	Boris
	Elif

S	
name	
	Anya
	Carmen
	Elif
	Diego

$R \cap S$ Difference $\rightarrow$ SQL: EXCEPT

Names in R that are NOT in S. Note: $R - S \neq S - R$!

Result: {Boris}



$R - S$ is NOT symmetric: $R - S \neq S - R$. Also: $(R \cup S) - (R \cap S)$ gives the symmetric difference.